

# Kohana — Lista tecnológica definitiva: de 0.9.4-beta a "el mejor agente early"

Basado en: lectura completa de `Kohana-0_9_4-tested-source.zip` (código, docs, roadmap), arquitectura pública de OpenClaw, incidentes de seguridad documentados, benchmarks 2026 de wake word / TTS / VAD / VLM, y las decisiones de producto que confirmaste (hardware auto-adaptativo, prioridad velocidad > voz > privacidad > integraciones, sí a memoria/subagentes/skills, no a puentes de mensajería).

## 0. Resumen ejecutivo

Kohana ya tiene la arquitectura correcta para llegar a donde quieres: separación limpia `Core`/`Windows`/`App`, Resource Governor real, política de privacidad de Vision, y un roadmap (0.10-1.0) que ya anticipa memoria controlable, permisos por skill y automatizaciones — es decir, ya está apuntando a corregir de fábrica los errores que hicieron famoso (y peligroso) a OpenClaw.

Lo que falta no es "más IA", es tres piezas de motor que hoy están resueltas con herramientas equivocadas:

| Pieza                   | Hoy (0.9.4)                                                                         | Problema real                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Wake word               | Vosk (ASR completo) forzado con gramática JSGF                                      | Vosk es un motor de dictado de vocabulario abierto disfrazado de spotter. Gasta CPU de más y falla más que un modelo hecho para esto. |
| Voz de salida           | <code>System.Speech.Synthesis</code> (SAPI5, ~2006)                                 | Es la razón por la que "no suena como Grok". No es un problema de prompt, es el motor.                                                |
| Fin de turno / silencio | Umbrales de amplitud escritos a mano (ver <code>VOICE_RELIABILITY_V1/V2.md</code> ) | Estás reinventando, con reglas manuales, lo que un modelo de 1.8 MB ya resuelve mejor.                                                |

El resto de este documento va a lo concreto: nombres exactos, licencias, por qué, y qué archivo de tu código reemplazan.

### Decisiones de producto ya confirmadas (esto es lo que hace "definitiva" a esta lista)

- **Hardware:** Kohana se auto-adapta a la máquina (3 niveles de capacidad), no asume ni PC modesta ni GPU potente por defecto. → Sección 3.0.
- **Prioridad, en este orden:** 1) velocidad/latencia, 2) calidad de voz e interacción natural, 3) privacidad 100% local, 4) amplitud de integraciones. Esto es una jerarquía de desempate real: cuando dos opciones técnicas compitan, gana la más rápida; entre dos igual de rápidas, gana la de voz más natural; solo si ambas empatan ahí entra "cuál es

más local"; la amplitud de integraciones nunca justifica sacrificar las tres anteriores.

- **De OpenClaw, sí quieres:** memoria a largo plazo tipo diario editable, agentes/subagentes especializados, y automatizaciones/skills instalables de terceros.
  - **De OpenClaw, explícitamente NO quieres:** controlar Kohana desde WhatsApp/Telegram/Discord. Esto es una decisión de arquitectura importante, no un detalle: los puentes de mensajería externa son, según los propios reportes de seguridad de OpenClaw, la puerta de entrada principal de la inyección de instrucciones (contenido no confiable que llega desde afuera y el modelo trata como orden). Al no construir eso, Kohana se queda automáticamente fuera de la mayor superficie de ataque que tuvo OpenClaw, sin tener que "arreglarlo" después. Si en el futuro quieres ese puente, que sea un skill opcional muy acotado (0.13, "servicios conectados"), nunca el canal principal de control.
- 

## 1. Diagnóstico del código actual

**Stack real detectado:** .NET 10, WPF (+ WinForms interop para tray icon), (NAudio 2.3.0), (Vosk 0.3.38), (Whisper.net 1.9.1), (System.Speech 10.0.10), arquitectura de proveedores de IA (OpenAI / Ollama / LM Studio / OpenAI-compatible) con runtime privado de Ollama administrado por la propia app.

### Lo que está bien hecho (no tocar todavía)

- (ResourceGovernorPolicy.cs): umbrales claros (GPU 88%, CPU 92%, RAM 92%), detección de pantalla completa, degradación a "Busy"/"Game" sin matar comandos locales. Esto es exactamente lo que le falta a la mayoría de agentes de escritorio — que consumen igual estés jugando o no.
- (VisionPrivacyPolicy.cs) + **captura bajo demanda en memoria**: nunca guarda en disco, bloquea gestores de contraseñas por nombre de proceso y por título de ventana. Esto te pone, de fábrica, del lado correcto de la controversia de **Windows Recall** (ver sección 6).
- (NaturalCommandParser.cs) + (SpanishCommandLexicon.cs): resolver comandos conocidos localmente antes de tocar el LLM es la decisión de latencia/costo más importante que existe en este tipo de apps, y ya la tomaste.
- **Separación** (IWakeWordService) / (IVoiceInputService) / (IVoiceOutputService): las interfaces ya están desacopladas. Reemplazar el motor de dentro es un cambio de implementación, no de arquitectura. Buenas noticias: todo lo de la sección 3 se conecta ahí sin romper nada.

### Los 3 cuellos de botella concretos

1. (VoskWakeWordService.cs): usa Vosk (motor de reconocimiento de voz continuo de propósito general) restringido con una gramática tipo lista ([ "oye kohana", "koana", ... ]) para simular un keyword spotter. Funciona, pero es la manera cara y menos precisa de resolver el problema — literalmente estás cargando un modelo acústico completo para detectar 3 palabras.

2. `WindowsTextToSpeechService.cs`: usa `System.Speech.Synthesis`, el SAPI5 de escritorio de Windows. Es la voz robótica de "Narrador" clásico. Ningún ajuste de prompt o de velocidad la va a acercar a Grok/ElevenLabs — es un techo de motor, no de configuración.
  3. **Segmentación de turno hecha a mano**: tus propios docs (`VOICE_RELIABILITY_V1.md`, `SHELL_VOICE_RELIABILITY_V2.md`) documentan que ajustaron manualmente "pausa menor a 1.5s no termina la orden", histéresis de ruido, etc. Esto es la definición de un problema que un **VAD neuronal** ya resuelve out-of-the-box y mejor.
- 

## 2. OpenClaw: qué es realmente y de qué alejarse

OpenClaw (antes Clawdbot/Moltbot) es un agente autónomo open-source (MIT) que pasó de 0 a ~250,000 estrellas de GitHub en semanas. Su arquitectura de referencia:

- **Gateway** como proceso único que gestiona sesiones (cola de comandos, un solo escritor por sesión).
- **Skills** = carpetas con un `SKILL.md` en lenguaje natural. El modelo solo ve **metadata** (nombre + descripción) hasta que decide leer el archivo completo — así no gasta contexto en instrucciones que no va a usar. *(Esto es, literalmente, el mismo patrón que usan los "Skills" de Claude — es un patrón de diseño validado, no una idea exótica.)*
- **Memoria por niveles**: Tier 1 (`MEMORY.md`, ~100 líneas, siempre cargado), Tier 2 (archivos diarios `YYYY-MM-DD.md`), Tier 3 (conocimiento profundo por persona/proyecto/tema), y una capa de **búsqueda semántica vectorial** (SQLite + embeddings) para encontrar lo que no está en las capas curadas.
- Conecta con WhatsApp, Telegram, Discord, Slack, Signal, iMessage — de ahí buena parte de su viralidad ("controla tu PC desde el celular").

### Por qué esto se volvió un problema de seguridad documentado

Esto es exactamente lo que debes evitar repetir, con fuente:

- Según reportes de seguridad (Blink/Ars Technica), una falla en el sistema de emparejamiento de dispositivos (bautizada como una de las vulnerabilidades críticas de OpenClaw, CVSS 9.8) permitía que cualquiera con el nivel de acceso más bajo se auto-otorgara acceso de administrador, porque el sistema nunca verificaba si quien aprobaba la solicitud tenía autoridad real para hacerlo.
- La inyección de instrucciones ("prompt injection") es el riesgo más documentado del ecosistema, según TechRadar y el propio equipo de OpenClaw: instrucciones maliciosas ocultas en un correo, documento o página web que el modelo trata como órdenes legítimas del usuario — el propio proyecto lo describe como un problema no resuelto a nivel de toda la industria. Cisco encontró que una auditoría de 31,000 skills de agentes reveló que el 26% contenía al menos una vulnerabilidad.
- Microsoft advirtió explícitamente que OpenClaw no es apto para correr en una estación de trabajo personal o empresarial estándar, y el regulador chino (MIIT/CNCERT) pidió auditorías de exposición de red y controles de acceso más robustos

tras detectar instancias mal configuradas.

- Esto **no es un accidente de un solo desarrollador descuidado**: es lo que pasa cuando un agente tiene ejecución de shell, navegación web y skills de terceros instalables sin sandbox real ni verificación de autoridad.

La buena noticia para ti

Tu roadmap 0.11 ("Planes de varias acciones antes de ejecutar", niveles `Bloqueado`/`Preguntar`/`Permitido`, "Manifiestos de skills con permisos, red y datos usados", "Registro de cada acción") **ya es, en papel, un diseño más seguro que el OpenClaw original**. El trabajo real es no diluir esos principios cuando llegue la presión de "hacer que funcione rápido".

3. Lista técnica por categoría

3.0 Detección automática de capacidad — la base de todo lo demás

Ya tienes la mitad de esta pieza: `ResourceGovernorPolicy.cs` ya distingue **carga dinámica** (Normal/Busy/Game, según CPU/GPU/RAM en el momento). Lo que falta es su complemento: una **clasificación de capacidad estática**, decidida una sola vez (al instalar o al iniciar), que responde "¿qué tan potente es este equipo?" en vez de "¿qué tan ocupado está ahora mismo?". Son dos ejes independientes y ambos deben existir:

| Eje                  | Pregunta que responde                           | Ya existe?                               |
|----------------------|-------------------------------------------------|------------------------------------------|
| Carga (dinámico)     | ¿Está el equipo ocupado <i>ahora mismo</i> ?    | Sí — <code>ResourceGovernorPolicy</code> |
| Capacidad (estático) | ¿De qué es <i>capaz</i> este equipo en general? | No — hay que construirlo                 |

Cómo detectarlo en .NET/Windows, con nombres exactos:

- VRAM y GPU dedicada: WMI (`Win32_VideoController`) o **Vortice.Windows** (bindings modernos y mantenidos de DirectX/DXGI para .NET) para consultar el adaptador y su memoria dedicada.
- Núcleos y arquitectura de CPU: `Environment.ProcessorCount` (rápido, ya disponible).
- RAM total: P/Invoke a `GlobalMemoryStatusEx`, o `Microsoft.VisualBasic.Devices.ComputerInfo.TotalPhysicalMemory`.

Con eso, Kohana clasifica el equipo una vez en **Ligero / Equilibrado / Premium**, y ese nivel decide el motor por defecto en cada subsistema (wake word siempre es prácticamente gratis en cualquier nivel; STT, TTS y VLM sí cambian):

| Nivel                                                    | STT                                                | TTS              | VLM local                                                                                        |
|----------------------------------------------------------|----------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------|
| Ligero (sin GPU, pocos núcleos)                          | Whisper <span>base</span> /<br><span>small</span>  | Piper            | Solo OCR nativo de Windows;<br>VLM se delega al proveedor en la nube si el usuario configuró uno |
| Equilibrado (CPU moderna, sin GPU dedicada o GPU básica) | Whisper <span>small</span> /<br><span>turbo</span> | Kokoro-82M       | SmolVLM2 local si no hay proveedor configurado                                                   |
| Premium (GPU NVIDIA 6GB+)                                | Whisper <span>turbo</span><br>acelerado por GPU    | Chatterbox-Turbo | Qwen3-VL vía Ollama, acelerado por GPU                                                           |

El usuario puede **forzar manualmente** un nivel distinto al detectado (por ejemplo, alguien con GPU que prefiere ahorrar batería en su laptop), pero el default siempre es automático. Esto también resuelve tu prioridad #1 (velocidad): nunca corres un modelo demasiado pesado para el hardware real de quien lo está usando.

**3.1 Wake word — reemplazar Vosk-como-spotter**

| Tecnología                                         | Licencia                        | Por qué                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| openWakeWord                                       | Apache 2.0                      | Framework de detección de wake word: convierte audio en espectrograma de Mel → embedding preentrenado (el modelo de embeddings de voz de Google, reutilizado) → un clasificador binario pequeño y propio por palabra de activación. En pruebas del propio proyecto, sus modelos incluidos <b>superan a Porcupine en precisión</b> con benchmarks públicos, y un solo núcleo de Raspberry Pi 3 corre 15-20 modelos simultáneos — en tu hardware de escritorio el costo es prácticamente cero. Corre vía <b>ONNX Runtime</b> , que ya tiene binding oficial y maduro para .NET ( <code>Microsoft.ML.OnnxRuntime</code> ), así que reemplaza <code>VoskWakeWordService.cs</code> sin tocar la interfaz <code>IWakeWordService</code> . |
| Pipeline de entrenamiento custom para "Oye Kohana" | —                               | El propio proyecto documenta cómo entrenar una palabra nueva sin grabar miles de voces reales: generar audio sintético con un TTS (decenas de voces distintas diciendo "Oye Kohana", "Kohana", "Hey Kohana"), aumentarlo con ruido de fondo y reverberación simulada (room impulse response), y entrenar el clasificador final sobre esos embeddings. Esto reemplaza directamente tu <code>BuildGrammar()</code> actual (la lista de variantes fonéticas escritas a mano tipo <code>"koana", "cohana", "kojana" ...</code> ) por un modelo que generaliza en vez de listar excepciones.                                                                                                                                             |
| Porcupine (Picovoice)                              | Comercial, capa gratis limitada | Más preciso aún en hardware muy restringido, pero requiere <code>AccessKey</code> y tiene límite de wake words personalizadas en el plan gratuito — no encaja con "gratis y open source al final". Mencionarlo solo como referencia de benchmark.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| microWakeWord                                      | Apache 2.0, TFLite              | Pensado para microcontroladores (ESPHome/Home Assistant). No aporta nada extra en un PC de escritorio frente a openWakeWord, que además ya es ONNX (más fácil de integrar en .NET que TFLite).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

### 3.2 VAD y fin de turno — la pieza que falta para el "talkback" real

| Tecnología                         | Licencia                      | Por qué                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Silero VAD                         | MIT, ONNX                     | Modelo dedicado de detección de actividad de voz: procesa audio en bloques de 32 ms a 16 kHz y devuelve una probabilidad de "hay voz humana" frame a frame. Corre en ONNX Runtime (mismo runtime que usarías para wake word — un solo motor de inferencia para ambas cosas). Esto <b>reemplaza directamente</b> la lógica de umbral de amplitud + histéresis manual de tus docs de "Voice Reliability".                                                                                                                                                                                                                                                                                                                      |
| Barge-in (interrupción con la voz) | — construir sobre lo anterior | Esto es lo que hoy le falta a Kohana para sonar como Grok/ChatGPT en modo voz: mientras Kohana está hablando, seguir corriendo Silero VAD sobre el micrófono; si detecta voz humana sostenida, cortar la síntesis ( <code>Stop()</code> ) ya existe en tu <code>IVoiceOutputService</code> ) y pasar a escuchar. Falta cancelación de eco ( <b>AEC</b> , Acoustic Echo Cancellation) para que Kohana no se "escuche a sí misma" por el micrófono y se corte sola — Windows expone esto vía <b>Windows Audio Session API (WASAPI) con Voice Isolation/AEC de sistema</b> en Windows 11; si no basta, la librería <b>WebRTC AEC3</b> (BSD) es la implementación de referencia que usa casi toda la industria de videollamadas. |

3.3 STT — Whisper ya es buena base, dos mejoras posibles

| Tecnología                                       | Por qué                                                                                                                                                                                                                                                                                |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mantener <b>Whisper.net</b> / <b>whisper.cpp</b> | Ya es una elección sólida y 100% local. Mejora de bajo esfuerzo: usar un modelo <i>distil</i> o el modo <i>turbo</i> de whisper.cpp para bajar la latencia sin perder tanta precisión como el actual <code>base</code> .                                                               |
| <b>Moonshine</b> (Useful Sensors, MIT)           | ASR diseñado específicamente para baja latencia en edge/CPU, pensado para frases cortas de comando de voz — el caso de uso exacto de "abre Visual Studio Code" o "qué es esto". Vale la pena evaluarlo como motor alternativo para órdenes cortas, dejando Whisper para dictado largo. |

3.4 Texto a voz (TTS) — el cambio de mayor impacto perceptible

Aquí está el salto de "suena a Windows XP" a "suena a un asistente de 2026". La clave no es un solo modelo: es una **arquitectura de 3 niveles**, porque el modelo más natural (Chatterbox) necesita GPU, y quieres que Kohana "consume poco" y funcione en cualquier PC.

| Nivel                             | Modelo                                      | Licencia   | Perfil                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------|---------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ligero (fallback universal)       | Piper                                       | MIT        | Corre en un solo núcleo de CPU, ~1 GB de RAM. Calidad más robótica ( $\approx 3.3$ MOS) pero tiene varias voces en español ( <code>es_ES</code> , <code>es_MX</code> , etc.). Ideal para modo ahorro de batería/PC modesta, y como voz de respaldo instantánea mientras carga un modelo mejor.                                                                                                                                                                    |
| Equilibrado (default recomendado) | Kokoro-82M                                  | Apache 2.0 | 82M de parámetros, corre en tiempo real en CPU pura. Incluye español, aunque con solo 3 voces (1 femenina, 2 masculinas) — su punto débil declarado es que los idiomas no ingleses tienen menos datos de entrenamiento, así que conviene probarlo bien en español antes de fijarlo como voz "oficial" de Kohana.                                                                                                                                                  |
| Premium (si hay GPU)              | Chatterbox / Chatterbox-Turbo (Resemble AI) | MIT        | La opción de mayor calidad: en pruebas ciegas publicadas por Resemble AI, el 65.3% de los oyentes prefirió Chatterbox-Turbo sobre ElevenLabs, contra 24.5% que prefirió ElevenLabs (10.2% neutral). Soporta español entre sus idiomas base y clonación de voz zero-shot (5-20 segundos de audio de referencia) — esta es la ruta técnica más realista para lograr <b>"una voz tan buena como la de Grok"</b> con licencia libre, pero exige GPU para tiempo real. |

**Detalle de arquitectura importante:** para que se sienta como una conversación real (no como "escribe todo, luego reproduce todo"), la síntesis debe hacerse **en streaming por oración/frase** — generar y reproducir el primer fragmento apenas está listo, mientras el resto todavía se está sintetizando. Esto reduce el "time to first audio" y es, más que el modelo en sí, lo que separa un asistente que "se siente instantáneo" de uno que se siente lento aunque use el mismo modelo.

### 3.5 Vision / "qué es esto" — ya vas bien encaminado, estas son las mejoras

- **OCR nativo de Windows (`Windows.Media.Ocr`):** es una API del propio sistema operativo, gratis, sin descargas ni modelos externos, y corre en CPU/NPU local. Extraer el texto de la ventana **antes** de decidir si hace falta mandar la imagen completa a un modelo de IA reduce tokens, baja la latencia y permite responder preguntas simples ("¿qué dice este error?") sin siquiera invocar un LLM. Tu propio `SHELL_V4_1_FOCUS_LOOK.md` ya lo tiene anotado como pendiente ("OCR local para decidir entre texto e imagen") — es más fácil de lo que parece porque ya es parte de Windows.



- **VLM local opcional** (para cuando el usuario no tiene proveedor de IA configurado, modo 100% offline): **SmolVLM2** (Apache 2.0, HuggingFace, pensado para edge) es la opción de licencia más segura; **Moondream2** es más ligero pero su licencia **restringe el uso comercial**, cuidado si Kohana eventualmente tiene un modelo de negocio; **Qwen3-VL** (variantes pequeñas) da mejor OCR multilingüe y ya encaja con tu arquitectura de proveedores vía Ollama.
- **UI Automation / IAccessible2 (árbol de accesibilidad de Windows)**: en vez de razonar solo sobre píxeles, leer los controles reales de la ventana activa (botones, texto, estado) es más preciso para responder y, más importante, es lo que te permite ejecutar **acciones reales** ("dale clic a Guardar") sin depender de coordenadas de píxel frágiles que se rompen con cualquier cambio de resolución o DPI. Esto es directamente relevante para tu roadmap 0.11 de "acciones aprobables".

### 3.6 Memoria — diario editable (confirmado como prioridad, para 0.10 "Kohana Runtime")

Estructura concreta recomendada, dentro de `%LocalAppData%\Kohana\Memory\`:

```
Memory/
  ESENCIAL.md           ← Tier 1: ~100 líneas, siempre en el prompt.
Curado (no crece solo)
  diario/
    2026-07-22.md       ← Tier 2: entradas del día, sin curar, en bruto
    2026-07-21.md
  conocimiento/
    personas/           ← Tier 3: una nota por persona relevante que
Kohana conoce
    proyectos/
    decisiones/
  indice.sqlite         ← Capa 4: embeddings para búsqueda semántica
sobre todo lo anterior
```

- Todo en **Markdown plano** — el usuario puede abrir `ESENCIAL.md` en el Bloc de notas y ver, literalmente, qué recuerda Kohana de él. Compatible con respaldo en Git si el usuario quiere.
- Búsqueda semántica (capa 4): un modelo de embeddings pequeño (`all-MiniLM-L6-v2` o equivalente, exportado a ONNX, corre en milisegundos en CPU) + `sqlite-vec` (sucesor libre y mantenido de `sqlite-vss`) como extensión vectorial de SQLite — sin servidor de base de datos vectorial pesado, todo en un archivo `.sqlite`.
- Comandos de voz directos sobre la memoria, ya con tu lexicón de comandos en español: "Kohana, olvida lo que sabes de mi trabajo", "exporta todo lo que recuerdas", "qué recuerdas de mí" — el control humano real que ya prometes en el roadmap 0.10, hecho verbo.
- Un archivo diario nunca se manda completo al LLM: solo `ESENCIAL.md` + los resultados

que la búsqueda semántica considere relevantes para la pregunta actual — así el "diario" puede crecer indefinidamente sin que cada conversación se vuelva más lenta (protege tu prioridad #1 de velocidad).

### 3.7 Subagentes especializados (confirmado, para 0.14)

La forma barata de implementar esto —sin violar "consume poco" ni la prioridad de velocidad— es **no** tratarlos como procesos o modelos separados, sino como **perfiles de configuración** sobre la misma conexión de IA ya establecida:

- Cada subagente = un `system prompt` propio + un subconjunto permitido de skills + su propia carpeta de memoria en `conocimiento/` + límites propios de tiempo/costo (ya contemplados en tu roadmap 0.14: "Delegación de tareas con límites de tiempo y costo").
- Patrón **orquestador-trabajador**: Kohana (el asistente principal) decide cuándo delegar ("esto es para el subagente de Programación") y el subagente responde dentro de su alcance; el resultado se anuncia de vuelta al hilo principal — el mismo patrón que usa OpenClaw para sus subagentes, pero sin necesitar spawnear procesos nuevos.
- Memorias y permisos **separados por agente** (ya en tu roadmap): el subagente de Estudio no debería poder leer la memoria del de Programación a menos que el usuario lo autorice explícitamente.
- Esto es, en esencia, una extensión natural de lo que ya hace `NaturalCommandParser` al enrutar: hoy enruta a "comando local vs. LLM", mañana puede enrutar a "LLM general vs. subagente especializado X".

### 3.8 Skills, permisos y acciones — marketplace de terceros (confirmado, 0.11-0.14)

Dado que sí quieres skills instalables de terceros (a diferencia de los puentes de mensajería, que descartaste), esta es la pieza donde más vale la pena aprender del error central de OpenClaw, en dos etapas:

- **Etap 1 (v1, segura)**: solo skills de primera mano, escritas y firmadas por el propio proyecto Kohana. Cero superficie de ataque de terceros todavía, pero ya se valida el formato de manifiesto y el sandbox.
- **Etap 2 (marketplace comunitario)**: recién ahí abrir a la comunidad, y solo con:
  - **Model Context Protocol (MCP)** como formato del manifiesto en vez de inventar uno propio — da compatibilidad de facto con un ecosistema que ya está creciendo, en vez de que Kohana quede aislada con su propio formato.
  - **Sandboxing real de ejecución** por skill (AppContainer de Windows, o un contenedor ligero) — no repetir el error #1 de OpenClaw: skills de terceros con el mismo nivel de acceso que el proceso principal.
  - **Verificación/firma antes de listar una skill como "confiable"** en cualquier catálogo — aprender del problema real de moderación que tuvo ClawHub (skills maliciosos empujados artificialmente al top).
  - Los tres niveles que ya tienes pensados (`Bloqueado`)/(`Preguntar`)/(`Permitido`) + registro de cada acción, aplicados también a lo que hace cada skill, no solo a las

### 3.9 LLM local y latencia percibida (prioridad #1 confirmada — esta sección es la que más importa)

- Ya cubres lo esencial (OpenAI / Ollama / LM Studio / compatibles). Complemento de bajo costo: un **clasificador de intención por embeddings** (ONNX, milisegundos) delante del LLM grande, para capturar comandos "casi conocidos" que hoy caen directo a una llamada de modelo completo — extiende lo que ya hace `NaturalCommandParser`, pero con tolerancia a variación de frase en vez de reglas exactas.
- **Reutilización de KV-cache / prompt caching** cuando el proveedor lo soporte, para que conversaciones largas no vuelvan a pagar el costo de latencia del contexto completo en cada turno.
- Dado que la velocidad quedó como prioridad #1, esto también debería influir el default del nivel "Premium": si hay GPU disponible, úsala primero para acelerar el LLM local (menor latencia que ir a la nube) y solo después para el TTS de mayor calidad, si ambos compiten por la misma GPU en el mismo instante.

### 3.10 Actualizaciones y distribución

- Firma de código (Authenticode) — ya está prevista en tu roadmap 1.0, correcto.
- **Velopack** (MIT, sucesor moderno y mantenido de Squirrel.Windows) para actualizaciones **delta** (solo bajar lo que cambió) en vez de reinstalar el paquete completo — importante para que las actualizaciones frecuentes de un proyecto activo no se sientan pesadas.

### 3.11 Animaciones "a la Apple" sin copiar a Apple ni a OpenClaw

- La clave técnica no es *qué tan bonito* sino *qué tipo de curva de animación*: usar **animaciones de resorte (spring physics)** en vez de solo curvas ease-in-out — es la diferencia entre movimiento que se siente "vivo" y movimiento que se siente "de PowerPoint". WPF permite `EasingFunction` personalizada; si en algún momento migras a WinUI 3, la **Composition API** de Windows ya trae animaciones de resorte nativas, más baratas en CPU que animar propiedades a mano.
- Sigue con tu propio lenguaje visual (Sakura Fluent, pétalos) — ya es distintivo y evita el problema de "parece un clon". El principio ya está en tus docs (`(VISUAL_FOUNDATION_V3.md)`): un solo movimiento dominante por interacción, opacidad + traslación, nunca escalar texto. Eso es, técnicamente, ya la filosofía correcta — el trabajo que falta es de ejecución, no de dirección.

---

## 4. Qué le falta a OpenClaw que Kohana ya debería tener de fábrica

1. **Verificación de autoridad real en cada aprobación**, no solo "¿tienes acceso?" — la falla de ClawJacked fue exactamente no chequear si quien aprobaba tenía permiso para aprobar.
2. **Sandbox real por skill**, no solo un manifiesto de permisos declarativo que un skill

malicioso puede ignorar.

3. **Auditoría/registro legible por humano de cada acción sensible**, no solo logs técnicos para debugging.
4. **Transparencia de red**: mostrar claramente cuándo algo sale a internet vs. cuándo todo quedó local — OpenClaw no distingue esto con claridad para el usuario final, y es una fuente enorme de la desconfianza que generó.
5. **Un "modo estación de trabajo segura" por defecto** (justo lo que Microsoft dijo que a OpenClaw le falta): permisos mínimos de fábrica, ampliables conscientemente, no al revés.

## 5. Qué sí vale la pena adoptar de OpenClaw

1. **Memoria en Markdown plano, no en una base de datos opaca** — el usuario puede abrir un archivo de texto y ver literalmente qué recuerda Kohana de él.
2. **Metadata-primero para skills**: el modelo ve nombre+descripción de cada skill, y solo carga el contenido completo de la que realmente necesita — ahorra contexto y dinero en tokens.
3. **Automatizaciones que se auto-explican en lenguaje natural** antes de ejecutarse (tu roadmap 0.12 ya apunta ahí).

---

## 6. Qué busca la comunidad (de foros, prensa y análisis técnico)

- **Miedo real y documentado a la sobre-recolección silenciosa**: el caso Windows Recall es la referencia obligada — según Gizmodo, investigadores de seguridad se preocuparon rápido porque el software era capaz de capturar información sensible como números de cuenta bancaria o de seguridad social, y no tardaron en encontrar fallas graves que permitían que cualquiera con acceso al equipo leyera los registros de capturas de pantalla de la IA. Kohana ya está del lado correcto (captura bajo demanda, en memoria, nunca en disco) — esto **debe ser un mensaje de marketing explícito**, no solo una decisión técnica interna.
- **Cansancio de la dependencia de nube y de las suscripciones**: la gente elige agentes que permiten "traer tu propia API key" o correr 100% local — ya lo tienes con Ollama/LM Studio.
- **Skills/automatizaciones como el enganche real**, pero con miedo justificado a instalar código de terceros sin saber qué hace — de ahí la importancia de sandboxing y manifiestos verificables, no solo declarativos.
- **Instalación de un clic, sin terminal**: tu propio roadmap 0.9.3 ya lo tiene como objetivo ("Descarga del modelo recomendado sin terminal") — sigue siendo, según toda la cobertura de OpenClaw, una barrera de entrada real para el usuario no técnico.

---

## 7. Priorización: qué atacar ya vs. qué puede esperar

Con tu orden confirmado (velocidad → calidad de voz → privacidad local → integraciones), el orden de construcción queda así:

## **Fase 1 — Motor de voz e infraestructura de velocidad (impacto inmediato, bajo riesgo):**

- Detección automática de capacidad (3.0) — sin esto, cualquier elección de modelo es una apuesta a ciegas sobre el hardware del usuario.
- Silero VAD reemplazando los umbrales manuales (mejora inmediata y medible de la detección de fin de frase, y habilita barge-in).
- openWakeWord reemplazando la gramática de Vosk (menos falsos positivos, menos CPU, responde más rápido).
- TTS de 3 niveles (Piper/Kokoro/Chatterbox-Turbo) con streaming por oración — es el cambio que el usuario *va a notar* de inmediato, y es donde vive tu prioridad #2 (calidad de voz).

## **Fase 2 — Lo que confirmaste que sí quieres de OpenClaw:**

- Memoria tipo diario editable (3.6) — es la base técnica que necesitan tanto los subagentes como las automatizaciones inteligentes.
- Skills de primera mano con manifiesto y sandbox (Etapa 1 de 3.8) — antes de pensar en marketplace comunitario.
- Subagentes especializados (3.7) — una vez que memoria y skills ya tienen su formato definido, delegar entre ellos es configuración, no arquitectura nueva.

## **Fase 3 — Correcto dejarlo para después (no te distraigas ahora):**

- Marketplace de skills de **terceros** (Etapa 2 de 3.8): espera a tener sandboxing real probado en producción con tus propias skills primero.
- Navegador aislado y emparejamiento de dispositivos (0.13-0.14) — tu propio roadmap ya los pone al final, y con razón: mayor superficie de ataque, menor urgencia para un v1 sólido.
- Puentes de mensajería externa (WhatsApp/Telegram/Discord): descartado como objetivo central por decisión tuya — si alguna vez se revisita, que sea la última pieza, no la primera.

---

## **8. Ideas adicionales para que Kohana sea "el mejor agente early"**

- **Modo "explica qué acabas de ver":** cuando Kohana usa Vision, mostrar brevemente qué detectó (texto vía OCR, controles vía UI Automation) antes de dar la respuesta — refuerza la confianza y hace visible que no "inventa".
- **Perfil de energía consciente de batería** (no solo de CPU/GPU): bajar automáticamente al nivel de TTS "Ligero" y pausar wake word en batería baja, no solo en modo Juego.
- **Exportación total de datos en un clic** ("llévate todo lo que sabes de mí") — refuerza exactamente la promesa de control humano que ya está en tu roadmap.
- **Modo invitado/temporal:** sesión sin memoria persistente para cuando alguien más usa tu PC.
- **Un panel de "qué salió de esta PC hoy":** lista simple de qué se envió a un proveedor

remoto y cuándo — la transparencia de red que le falta a OpenClaw, convertida en feature visible.

---

## 9. Notas de licencia (para que "gratis y open source" no tenga sorpresas)

- MIT / Apache 2.0 sin restricciones prácticas: openWakeWord, Silero VAD, Piper, Kokoro-82M, Chatterbox, SmolVLM2, Velopack, sqlite-vec.
- Cuidado: **Moondream2** restringe uso comercial — evaluar solo si Kohana se mantiene estrictamente no comercial, o buscar alternativa (SmolVLM2/Qwen3-VL) si en algún momento hay un modelo de negocio.
- Porcupine es útil solo como referencia de benchmark, no como dependencia (licencia comercial con clave de acceso).